Build Instructions for Qwen2-VL Integration

Prerequisites

Development Environment

- **Xcode**: 15.0 or later
- **iOS Deployment Target**: 15.0+
- **Swift**: 5.9+
- **macOS**: 13.0+ (for MLX support)

Hardware Requirements

- **Development**: Mac with Apple Silicon (M1/M2/M3) for MLX testing
- **Target Device**: iPhone/iPad with A12 Bionic or newer
- **Storage**: 4GB free space for development
- **RAM**: 16GB recommended for development machine

Setup Instructions

1. Clone and Setup Repository

```
git clone https://github.com/ricobiz/IOSeiros.git
cd IOSeiros
git checkout qwen2vl-integration
```

2. Install Dependencies

Option A: Using Swift Package Manager (Recommended)

The project will automatically resolve MLX dependencies when building on Apple Silicon.

Option B: Manual MLX Installation (if needed)

```
# Install MLX Python tools for model conversion (optional)
pip install mlx-lm mlx-vlm

# For development and testing
pip install transformers torch pillow
```

3. Xcode Configuration

Project Settings

1. Open `EirosAgent.xcodeproj` in Xcode
2. Select the project target
3. Set **Deployment Target** to iOS 15.0+
4. Enable **Metal** framework in Build Phases
5. Add **MLX** framework dependency (auto-resolved)

Build Settings

```
SWIFT_VERSION = 5.9
IPHONEOS_DEPLOYMENT_TARGET = 15.0
ENABLE_BITCODE = NO
METAL_ENABLE_DEBUG_INFO = YES (Debug only)
```

Capabilities

- **App Sandbox**: Disabled (for file access)
- **Hardened Runtime**: Enabled
- **Metal**: Enabled

4. Model Setup

Automatic Download (Recommended)

The app will automatically download the Qwen2-VL-2B model on first run:
- Model size: ~1.2GB
- Download location: App Documents directory
- Requires internet connection for initial setup

Manual Model Installation (Optional)

```
# Download model manually (for development)
python -c "
from mlx_vlm import load
model, processor = load('mlx-community/Qwen2-VL-2B-4bit')
print('Model downloaded successfully')
"
```

Building the Project

1. Clean Build

```
# Clean derived data
rm -rf ~/Library/Developer/Xcode/DerivedData/EirosAgent-*

# In Xcode: Product → Clean Build Folder
```

2. Build Configuration

Debug Build

- Includes verbose logging
- Model performance metrics enabled
- Fallback to mock data if model fails
- Simulator support (limited functionality)

Release Build

- Optimized performance
- Reduced logging
- Production error handling
- Device-only (MLX requires Apple Silicon)

3. Build Commands

```
# Command line build (optional)
xcodebuild -project EirosAgent.xcodeproj -scheme EirosAgent -configuration Debug -des-
tination 'platform=iOS Simulator,name=iPhone 15 Pro'

# For device
xcodebuild -project EirosAgent.xcodeproj -scheme EirosAgent -configuration Release -
destination 'generic/platform=iOS'
```

Testing

1. Unit Tests

```
# Run unit tests
xcodebuild test -project EirosAgent.xcodeproj -scheme EirosAgent -destination 'plat-
form=iOS Simulator,name=iPhone 15 Pro'
```

2. Integration Tests

- Test vision analysis with sample images
- Verify model loading and unloading
- Check memory usage and performance
- Validate error handling scenarios

3. Device Testing

- Install on physical device for MLX testing
- Test with various image types and sizes
- Monitor thermal performance
- Validate battery impact

Troubleshooting

Common Build Issues

1. MLX Framework Not Found

```
Error: No such module 'MLX'
```

Solution:
- Ensure building on Apple Silicon Mac
- Update to latest Xcode
- Clean and rebuild project

2. Metal Compilation Errors

```
Error: Metal shader compilation failed
```

Solution:
- Enable Metal framework in project settings

- Check iOS deployment target (15.0+)
- Verify device supports Metal

3. Model Download Failures

```
Error: Failed to download Qwen2-VL model
```

Solution:
- Check internet connection
- Verify available storage space
- Clear app data and retry

4. Memory Issues

```
Error: Memory pressure, model unloaded
```

Solution:
- Close other apps
- Restart device
- Use 4-bit quantized model (default)

Performance Optimization

1. Build Optimizations

```swift
// In release builds, disable debug features
#if DEBUG
    let enableVerboseLogging = true
    let enablePerformanceMetrics = true
#else
    let enableVerboseLogging = false
    let enablePerformanceMetrics = false
#endif
```

2. Memory Management

- Monitor memory usage during development
- Use Instruments to profile performance
- Test on devices with different RAM amounts
- Implement memory pressure handling

3. Thermal Management

- Monitor device temperature during testing
- Implement thermal throttling
- Reduce inference frequency if overheating
- Provide user feedback for thermal issues

Deployment

1. App Store Preparation

Bundle Configuration

```
<!-- Info.plist additions -->
<key>NSCameraUsageDescription</key>
<string>Camera access for real-time vision analysis</string>

<key>NSPhotoLibraryUsageDescription</key>
<string>Photo library access for image analysis</string>

<key>MLModelUsageDescription</key>
<string>On-device AI model for privacy-preserving vision analysis</string>
```

Privacy Manifest

```
{
  "NSPrivacyCollectedDataTypes": [],
  "NSPrivacyTrackingDomains": [],
  "NSPrivacyAccessedAPITypes": [
    {
      "NSPrivacyAccessedAPIType": "NSPrivacyAccessedAPICategoryUserDefaults",
      "NSPrivacyAccessedAPITypeReasons": ["CA92.1"]
    }
  ]
}
```

2. Distribution Options

Option A: Model Bundled with App

- Larger app size (~1.5GB total)
- Immediate functionality
- No internet required after install
- Longer App Store review time

Option B: Model Downloaded on Demand

- Smaller initial download (~300MB)
- Requires internet for first use
- Faster App Store approval
- Better user experience for updates

3. Version Management

```
struct ModelVersion {
    static let current = "1.0.0"
    static let minimumSupported = "1.0.0"
    static let updateURL = "https://api.example.com/models/qwen2vl"
}
```

Continuous Integration

1. GitHub Actions (Example)

```yaml
name: Build and Test
on: [push, pull_request]

jobs:
  build:
    runs-on: macos-14
    steps:
    - uses: actions/checkout@v4
    - name: Build
      run: xcodebuild -project EirosAgent.xcodeproj -scheme EirosAgent build
    - name: Test
      run: xcodebuild -project EirosAgent.xcodeproj -scheme EirosAgent test
```

2. Automated Testing

- Unit tests for all managers
- Integration tests for vision pipeline
- Performance benchmarks
- Memory leak detection

3. Quality Assurance

- Code coverage reports
- Static analysis with SwiftLint
- Performance regression testing
- Device compatibility validation

Maintenance

1. Model Updates

- Monitor for new Qwen2-VL releases
- Test compatibility with app
- Implement gradual rollout
- Maintain backward compatibility

2. Performance Monitoring

- Track inference times
- Monitor memory usage patterns
- Analyze crash reports
- User feedback integration

3. Documentation Updates

- Keep build instructions current
- Update troubleshooting guides
- Maintain API documentation
- Version release notes

Support

Development Support

- Check GitHub issues for known problems
- Review integration documentation
- Test on multiple device types
- Monitor performance metrics

Production Support

- Implement crash reporting
- User feedback collection
- Performance analytics
- Remote configuration for model parameters

For additional support or questions about the Qwen2-VL integration, please refer to the main documentation or create an issue in the repository.